DNDS5020 – Data Management and Databases
Winter 2026

Instructors: Jascha Grübel & Munther Ahmed

Final Report

Group H

Unifying CEU's Course Information System:
A Student-Centered Database Redesign

Group members

David Diestre Rubio
Teodore Giorgobiani
Thanika Haltrich
Talha Sadiq

April 12, 2026

1 Introduction

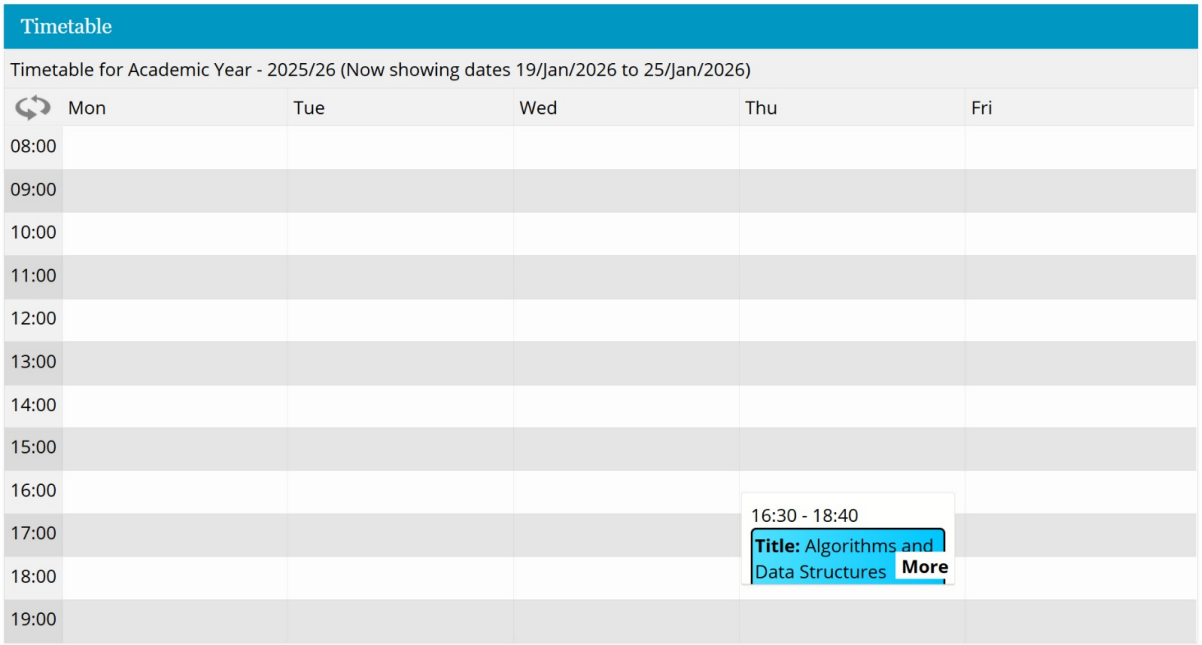


Figure 1: David's timetable illustrating the single-course visibility issue in the current process.

When looking at the timetable from SITS, David found that he had just one class per week, and although he did come to Vienna on exchange, he does not intend on partying *that* much. This is CEU's course database problem. In David's case, since he is part of the BA department, the courses in the department of Network and Data Science do not show in his timetable. Thanika (also an undergraduate) experienced a similar problem.

We wondered how we can improve upon this system and simplify it so students can register for courses, see their schedules, instructors and grades in one place to make the registration experience as convenient as possible. This was our starting point. Over the course of this report we will go into detail how we worked towards this goal.

1.1 Motivation and problem statement

At CEU, students often need to check multiple systems to understand what they can take, where classes are held, and whether courses satisfy their program rules. A 2025 student survey highlighted dissatisfaction with fragmented course information and the need to unify data into one coherent platform [1]. A second survey from 2026 reached similar conclusions regarding usability and planning burden [2].

The core issue is data fragmentation across independent tools (SITS, TimeEdit, Airtable), where each source is made for a different purpose. CEU relies on these heterogeneous systems rather than an end-to-end student planning platform. By contrast, unified student intranet systems such as that of Universitat Rovira i Virgili' s (David's home university) demonstrate that consolidated academic information significantly reduces decision friction for students [3]. David himself has stated that the course registration system at URV was easier compared to CEU.

1.2 Project objective and scope

Our objective is to redesign CEU course data management as a unified PostgreSQL relational database that supports student-facing and administrative workflows in a single schema. Specifically, we target: (i) course discoverability across departments, (ii) prerequisite-aware request handling, (iii) degree-progress credit checks, and (iv) timetable-aware planning and rescheduling support. These goals have directly influenced our decisions for making our example views and queries that can be seen in sections 3.4 - 3.6.

2 Background on data and problem

2.1 Why this is a data management problem

From section 1, we have highlighted the main challenge of the CEU course information system - heterogeneity and fragmentation (information is dispersed throughout many platforms, and updates are not consistent throughout the platforms). However, the different platforms rely on the same data from CEU. Therefore, we seek to build a new RDMS with this data.

2.1.1 Role of ACID properties in addressing data fragmentation

Using a centralized relational database implies incorporating ACID principles, which can directly assess issues mentioned above in the following way:

Atomicity ensures that transactions are executed in an all-or-nothing way. In our case, when creating or updating a course, all related data (e.g., timetable, instructor, credits) is either fully recorded or not applied at all, preventing partial and inconsistent entries across systems.

Consistency enforces that all data follows predefined rules and constraints. This includes standardized course codes, valid prerequisite formats, and correct mappings between courses and degree levels, eliminating inconsistencies across platforms. This would be very helpful for CEU since the current data is rather inconsistent. This has given rise to issues such as difficulties in mapping courses to prerequisites (see section 3 example query 2)

Isolation ensures that concurrent operations do not interfere with one another. In practice, this prevents conflicts during simultaneous course registrations or updates, ensuring students only see complete and stable course data.

Durability guarantees that once a transaction is committed, it is permanently stored. This ensures that course registrations and administrative updates are not lost due to system failures.

2.1.2 Structure of Data

A relational database also fits this problem because the data is structured around entities with clear relationships (courses connect to instructors, departments, programs, and students). Taking everything into account, our new database design contains 17 tables - 5 core entities and 9 junction tables to handle many-to-many relationships. The full schema can be seen in figure 2, section 3.1

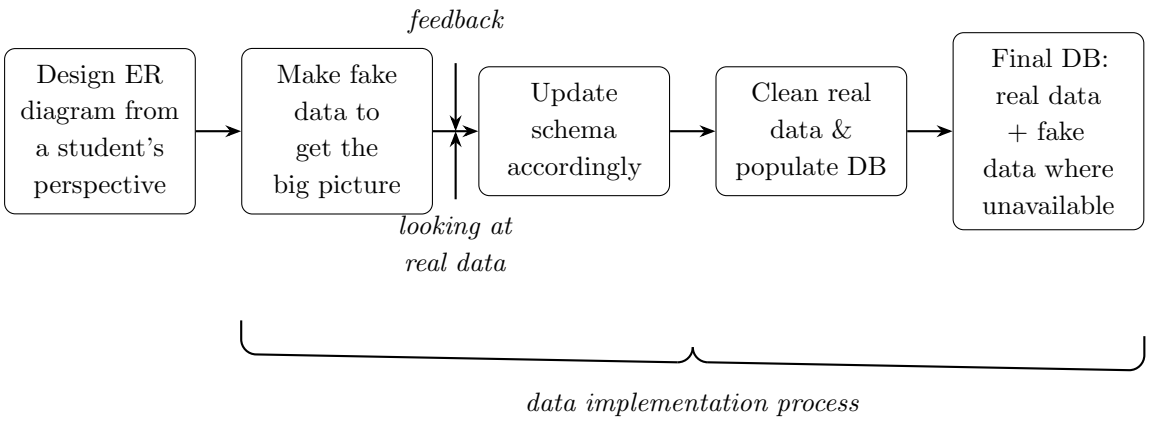
Entity	What it represents
Course	A course with code, department, and exclusion rules
Instructor	A faculty member who teaches courses
Department	An academic unit that offers courses and houses instructors
Program	A degree program with required and elective courses
Student	A student enrolled in a program

Table 1: The core entities of our schema

The relationships between these entities require junction tables; some even need multiple depending on the relation: TEACHING_COURSE links instructors to courses; DEPARTMENT_INSTRUCTOR associates instructors with departments; the tables STUDENT_REQUESTED_ENROLLMENT_IN_COURSE (pending requests), STUDENT_ENROLLED_IN_COURSE (enrollment), and STUDENT_PASSED_COURSE (completed courses) are key for different stages of the student-course interaction; and the three program-course tables (PROGRAM_MANDATORY_COURSE, PROGRAM_ELECTIVE_COURSE, PROGRAM_MANDATORY_ELECTIVE_COURSE) capture different types of degree requirements.

2.2 Workflow and problems with using real world data

The workflow we had was the following:



In the final database, the data we have used is the following:

- CEU course data from 2025 [4]
- Updated courses for 2026 [5]
- Scheduling data from 2025 and 2026 [6]
- Fake student data (used to illustrate the functions of our database which will be shown in section 3)
- Fake data on program learning outcomes (incorporated based on feedback we received from faculty) (we were not able to implement real data since it is dispersed on web pages and would be outside the scope of this course) [7]

During this workflow, some problems arose during the data implementation process. The original CEU course data had significant quality and structural issues, including non-normalized attributes, inconsistent and poorly formatted text data (which in turn caused CSV parsing issues), and a high proportion of missing values (approximately 56%). Additional challenges came from encoding errors and invalid data formats such as incorrect date and boolean representations. Furthermore, the real data did not fully align with our initial schema (which was designed on the assumption of perfectly cleaned data). Some key attributes were missing or dispersed, while others did not exist in the schema (e.g., `course_type`). Lastly, we experienced constraint violations, including missing values in non-nullable fields and duplicate records.

3 Methodology and implementation

3.1 ER design and relational mapping

We started with a conceptual ER model and iteratively refined constraints and cardinalities during implementation. The final model was drawn with `dbdiagram.io` [8] and then implemented in PostgreSQL.

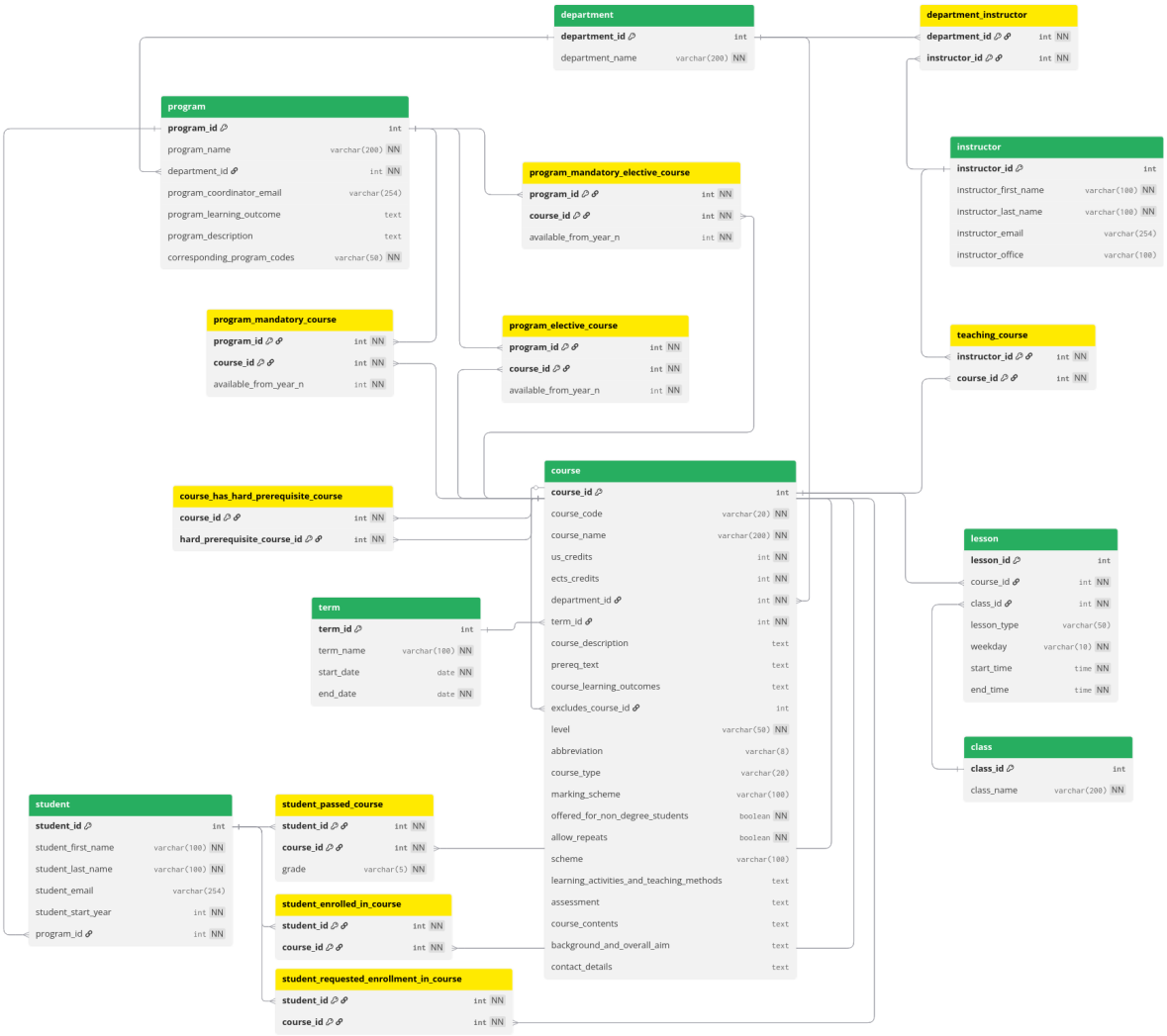


Figure 2: Final ER model used in implementation (Week 12 project materials).

The core entities (green) are: `department`, `program`, `instructor`, `student`, `term`, `course`, `class`, `lesson`. Junction tables (yellow) represent relations such as teaching course, department affiliation, enrollment lifecycle, hard prerequisite dependencies, and different types of course in a program.

3.2 Schema design choices and data integrity

Some technical design choices were made to keep the schema clear, normalized, and practical for queries. Instead of storing all relationships as simple attributes in a single junction table, in some cases we used multiple junction tables where each one represented a different relationship. For example, `student_requested_enrollment_in_course`, `student_enrolled_in_course`, and `student_passed_course` separate request, current enrollment, and course completion, avoiding mixed states in one table. The same idea was used for curriculum structure with `program_mandatory_course`, `program_elective_course`, and `program_mandatory_elective_course`.

Another important choice was to model timetable information through `lesson` and `class` instead of adding schedule fields directly to `course`. This allows one course to have multiple sessions, rooms, and lesson types, and makes the timetable view easier to derive from enrollments. A few indexes were also added on `course(department_id)`, `course(term_id)`, and `student(program_id)` because these are common join and filtering columns. Basic integrity is still enforced in the database itself through primary keys, foreign keys, composite keys, uniqueness constraints, and checks such as valid weekdays and `end_time > start_time`, as shown in listing 1 in the Appendix.

3.3 Data loading workflow, make commands, and reproducibility

To make database creation, loading, and later modifications easier to repeat, the project uses a small `Makefile` as a wrapper around `psql`. This avoids rewriting long execution commands every time and keeps the workflow consistent across development and testing, using the global variables defined in the `.env` file for passwords, connection details, and schema name.

Within this workflow, `make schema` applies `sql/01_schema.sql` to create tables, constraints, and indexes; `make load` runs `sql/02_load.sql` to import the CSV files inside the `dump_folder`; and `make views` runs `sql/03_views.sql` to create or refresh the timetable view. `sql/00_reset.sql` supports repeated rebuilds: `make reset-drop` removes the project tables, `make reset-truncate` clears data but keeps the structure, and `make reset-view` drops only the materialized view. Finally, `make queries` runs the example query file. The loader also checks whether each CSV file exists before importing it, so missing files are skipped and the process remains reproducible.

3.4 Views

The current implemented view is a materialized timetable view that consolidates enrollment and lesson data. The full definition is provided in listing 2 in the Appendix.

The view consolidates a student's weekly timetable by joining `student_enrolled_in_course`, `course`, `lesson`, and `class`. The decision to make it materialized comes from the fact that students' timetables are rarely changed but are frequently read. Pre-computing and storing the result avoids repeated joins. It can be refreshed with `REFRESH MATERIALIZED VIEW` when enrollment data changes.

Output includes columns of weekday, course (code + name), time slot, lesson type, and room name. Results are ordered first by weekday (Monday to Friday), then by start time.

Results

	weekday	course	time	lesson_type	room
1	monday	INTR5064 - International Political Economy of Digital Everything	12:45:00 - 14:30:00	seminar	QS D-103
2	tuesday	DOPP5305 - Mobilizing Contention? Social Movements as Democratic Actors	17:40:00 - 20:25:00	lecture	QS A-516
3	wednesday	DOPP5671 - Tackling Contemporary Policy Issues: Seminar	19:30:00 - 21:10:00	lecture	QS A-211
4	friday	UGST4188 - Fundamentals of Documentary Photography	12:30:00 - 13:45:00	seminar	QS D-108

This is the timetable for the student whose `student_id`=2. It can be seen that the student is able to see which days she has what class, along with the time of class, the lesson type, and the room the lesson is being held.

3.5 Queries

The following section displays representative query groups that map directly to user stories.

Query 1: Available courses for a student. The full SQL is shown in listing 3 in the Appendix. It describes a situation when a student wants to see which courses are available for them to take. The full query uses three `UNION ALL` blocks, one per course category. The excerpt shows the mandatory block; the elective and mandatory-elective blocks are structurally identical but join `program_elective_course` and `program_mandatory_elective_course`, respectively. Each block also applies a year gate filter.

Results

	course_code	course_name	ects_credits	term_name	course_type	available_from_year_n
1	DOPP5069	The Political Economy of Non-democracies	4	Spring Term	Elective	1
2	DOPP5070	The Politics of South-South Development in	4	Spring Term	Elective	1
3	DNDS5015	Network Science	4	Fall Term	Mandatory	1
4	PHIL5650	Ethics	4	Fall Term	Mandatory	1
5	DNDS5016	Applied Statistics	4	Fall Term	Mandatory Elective	2
6	DNDS5020	Data Management and Databases	4	Spring Term	Mandatory Elective	1

Figure 3: This is a sample of the available mandatory, mandatory-elective, and elective courses for a student. Furthermore, the course credits, the term the course is offered, and from what year the course will be available to her is also shown.

Query 2: Prerequisite decision support. The purpose of this query is is for simplifying the course registration process. The full SQL is shown in listing 4 in the Appendix. For each pending request in `student_requested_enrollment_in_course`, it `LEFT JOIN`s to `course_has_hard_prerequisite_course`, then checks if the student has passed or is enrolled in all hard prerequisites. The `CASE` expression produces a decision label. Key techniques include self-join on `course` (to get prerequisite names via `STRING_AGG`), `COUNT` comparison logic (if count of satisfied prereqs = count of required prereqs, auto-approve), and `COALESCE` for soft prerequisite display.

Results

	student_name	course_code	course_name	hard_prerequisites	soft_prerequisites	decision
1	Mia Corduneanu-Huci	UGST4085	Numerical Methods in Calculus and Algebra	Introduction to Linear Algebra, Intı	Prerequisites: Introduction to Linear Alge	AUTO-APPROVE
2	Lisa Adams	GENS5049	Qualitative Methods in Social Science Resear	None	none	AUTO-APPROVE
3	Lisa Adams	UGST4068	Social Policy and Labour Markets	None	First year courses in microeconomics	AUTO-APPROVE
4	Lisa Adams	POLS5156	Political Theory: Justice and Equality	None	This course is open to all MA students.	AUTO-APPROVE
5	Lisa Adams	INTR5453	Critical Approaches to Strategy and Security	None	There are no course prerequisites. Althou	AUTO-APPROVE
6	Diana Foster	DOPP5305	Mobilizing Contention? Social Movements as	Law and Public Policy, Technology	none	FLAG: prerequisites not met
7	Joseph Bowers	DOPP5305	Mobilizing Contention? Social Movements as	Law and Public Policy, Technology	none	FLAG: prerequisites not met
8	Lisa Adams	DOPP5675	Degrowth and Public Policy	Economic Analysis for Public Polic	[NULL]	FLAG: prerequisites not met
9	Nancy Baker	UGST4085	Numerical Methods in Calculus and Algebra	Introduction to Linear Algebra, Intı	Prerequisites: Introduction to Linear Alge	FLAG: prerequisites not met
10	Richard Brown	POLS5136	Experimental Political Science	Introduction to Social Science Sta	The class will not focus on data analysis/	FLAG: prerequisites not met

This is a sample of enrollment requests. It can be seen that when a student meets a courses' hard prerequisite (the first row), or when a course has no hard prerequisite (rows 2 to 5), the request is automatically approved. When these conditions are not met (rows 6 to 10) the request gets flagged. The idea of this query is that the flagged requests will be reviewed by a staff at CEU (in the case that the student had previously taken a similar course or for exchange students). An important note for this query: ideally we would not need the `soft_prerequisites` column and all the prerequisit requirements are stored in the `hard_prerequisites` column. However, since most of the prerequisite information we obtained from the raw CEU data was not formatted and was typed by professors manually, it was impossible to obtain prerequisites for each course without splitting it into hard and soft prerequisites.

Query 3: Credit progress for one student. The full SQL is shown in listing 5 in the Appendix. It shows a case where a student or an advisor wants to see the number of credits covered and remaining. The query joins `student` on `program` on `program_mandatory_course` on `course`, then `LEFT JOIN`s to `student_passed_course`. It uses `SUM` with the `FILTER (WHERE ...)` clause to separately sum total vs. passed ECTS in a single aggregation.

Results

	student_name	program_name	total_mandatory_ects_credits	passed_mandatory_ects_credits	remaining_mandatory
1	Mia Corduneanu-Huci	BA in Data Science and Society (4 years)	26	4	22
2	Leslie Adams	BSc in Data Science and Society (3 years	18	0	18
3	Lisa Adams	MA in Historical Studies (1 year) - History	32	0	32

This is a sample output for some students. It can be seen that for each student, this query shows the amount of mandatory credits needed to complete the program, how many credits the student has acquired, and how many more the student needs to obtain. However, this query can be modified to show the credit accumulation of (mandatory) elective courses also. This flexibility is important since CEU students need to accumulate a certain amount of credits for all mandatory, mandatory-elective, and elective courses.

3.6 Complex query 4 (rescheduling): idea and summarized code

Idea. Query 4 finds common free time windows for all students enrolled in a target course. The use case is operational: if a class needs to be rescheduled or a make-up session is needed, the system should suggest when all affected students are available and which rooms are free. A summarized version of the query is provided in listing 6 in the Appendix.

The query works in stages using ten chained CTEs (Common Table Expressions). First, it identifies all students enrolled in the target course and collects their occupied lesson intervals across all courses they attend. It clips these intervals to a standard working-day range (08:20 to 19:20) and merges overlapping or touching busy blocks on each weekday. Finally, it computes the complementary free intervals and checks room availability for each slot.

CTE walkthrough.

1. `params` sets the target course as an input parameter.
2. `weekday_order` provides a static Monday-to-Friday lookup so results are sorted by weekday, not alphabetically.
3. `course_students` retrieves students enrolled in the target course.
4. `occupied_lessons` collects all lesson intervals for those students (across all their courses) and clips them to the 08:20 to 19:20 window using `GREATEST` and `LEAST`.
5. `running_busy` computes a running maximum of `busy_end` using a window function (`MAX OVER`), which prepares the data for interval merging.
6. `busy_groups` detects where a new non-overlapping interval begins by comparing each row's start to the previous running end via `LAG`. A cumulative `SUM` assigns a group identifier to each connected block of busy time. This is an instance of the well-known "gaps and islands" pattern.
7. `merged_busy` collapses each group into one interval using `MIN(start)` and `MAX(end)`.
8. `day_bounds` defines the 08:20 to 19:20 boundary for each weekday.
9. `busy_with_sentinel` appends a sentinel row at the end of each day so that the last free slot can be captured.
10. `free_slots` uses `LAG` to find the gaps between consecutive merged busy blocks. Each gap is a candidate free window.

The final `SELECT` returns the weekday, start and end of each free window, its duration, and a correlated subquery that lists all rooms not occupied during that window using `NOT EXISTS`.

Key SQL techniques. This query relies on window functions (`MAX OVER`, `LAG`, cumulative `SUM`) to implement interval merging without procedural code. The `GREATEST/LEAST` functions handle interval clipping. The correlated subquery for room availability runs once per output row and checks all rooms against the lesson schedule. The full implementation spans roughly 150 lines and is available in the project repository under `sql/queries/example_queries.sql`.

Results

	course_name	weekday	free_start	free_end	free_duration	rooms_available
1	Introduction to Statistics	monday	09:20:00	10:45:00	01:25:00	QS A-201, QS A-211, QS A-215, QS A-
2	Introduction to Statistics	monday	12:45:00	19:20:00	06:35:00	QS A-101, QS A-103, QS A-214, QS B-
3	Introduction to Statistics	tuesday	08:20:00	19:20:00	11:00:00	QS A-018 Comp Lab, QS C-207, QS D
4	Introduction to Statistics	wednesday	08:20:00	09:15:00	00:55:00	N13 118 Tiered, N13 307/A, QS A-018
5	Introduction to Statistics	wednesday	17:00:00	19:20:00	02:20:00	N13 118 Tiered, N13 301, N13 517, QS
6	Introduction to Statistics	thursday	08:20:00	19:20:00	11:00:00	QS A-019 Comp Lab, QS A-201, QS A
7	Introduction to Statistics	friday	08:20:00	09:15:00	00:55:00	N13 301, N13 517, QS A-019 Comp La
8	Introduction to Statistics	friday	17:00:00	19:20:00	02:20:00	N13 301, N13 307/A, QS A-201, QS A-

These are the times available for rescheduling the course 'Introduction to Statistics' based on the availability of students enrolled in this course. It can be seen that the query recommends slots from Monday through Friday and shows the duration of each free slot (free_duration). Furthermore, it also shows rooms available for each free slot.

3.7 Public GitHub repository

The codebase and SQL artifacts are available at:

https://github.com/DavidDiestreR/CEU-DMDB-GroupH.git

3.7.1 Private data handling

Since we were explicitly asked not to publish private CEU data, we introduced a small set of repository workflows aimed at preventing data leakage while still allowing the project to remain publicly accessible. In particular, we created a `private` data folder for sensitive files and kept it excluded from version control. The same policy was applied to the `dump_folder`, so that data files moved there for loading into the database cannot be uploaded to the public repository by mistake. Both directories are therefore ignored through the project's `.gitignore` configuration.

In addition, environment-specific variables and credentials were not stored directly in the repository. Instead, they were handled through a local `.env` file, while only a safe `.env.example` template was shared publicly. This ensures that connection details and other sensitive configuration values remain outside the published codebase.

Finally, because Jupyter notebooks can unintentionally expose data through executed cells, we also introduced a notebook hygiene workflow. Before notebooks are committed, their outputs are stripped, and the same rule is validated again in the repository workflow. This reduces the risk of accidentally publishing loaded cells for data preprocessing notebooks with fragments of private CEU data inside notebook files.

4 Discussion

4.1 What was difficult

- **Prerequisite normalization.** Source data describes prerequisites as free text (e.g., "Recommended: Introduction to Statistics"). Only a subset of these could be encoded as foreign key edges in `course_has_hard_prerequisite_course`; the remainder is stored as advisory text in `prereq_text`. This is a limitation of the source data, not of the schema, but it constrains how much the system can automate.
- **Cross-source field mapping.** The CEU course dataset (2025) and the TimeEdit schedule export (2026) use different identifiers, naming conventions, and time formats. Reconciling equivalent fields required iterative cleaning in both SQL (`data_preprocessing/1nf.sql`) and Python (`data_preprocessing/generate_csvs.py`), with several manual judgment calls on how to match records.
- **Interval merging in SQL.** Query 4 required implementing the "gaps and islands" pattern with ten CTEs and multiple window functions. Writing and debugging this purely in SQL was significantly more complex than anticipated, especially for handling edge cases such as adjacent intervals and day boundaries.
- **Schema generality versus readability.** Early iterations used more abstract designs (e.g., a single program-course junction table with a type column). We moved to three separate tables (`program_mandatory_course`, `program_elective_course`, `program_mandatory_elective_course`) because the explicit structure was easier to query and easier to explain to non-technical stakeholders.

4.2 What did not work as expected

- **Partial prerequisite automation.** We originally aimed for near-complete automatic prerequisite validation at enrollment time. In practice, only hard prerequisites (explicit course-to-course dependencies) could be verified programmatically. Soft prerequisites remain human-readable text that the system can display but cannot enforce.
- **Materialized view parameterization.** The `student_timetable` view (see listing 2 in the Appendix) is hardcoded to a single student. PostgreSQL does not support parameterized materialized views, so serving multiple students would require either a function-based approach or a regular view with filtering at query time. For the prototype this was acceptable, but it would not scale to production use without modification.
- **Synthetic data limitations.** Real student enrollment records were not available, so parts of the dataset (enrollment, passed courses, enrollment requests) rely on generated dummy data. This means query outputs cannot be validated against institutional ground truth, only against internal consistency.

4.3 Lessons learned

- **Reproducibility infrastructure pays off early.** The Makefile, staged SQL scripts (00 through 03), and deterministic CSV generation made it possible to reset and rebuild the entire database in a single command. This saved considerable debugging time and made collaboration across the team straightforward.
- **An ER diagram is necessary but not sufficient.** The schema only proved itself when we loaded realistic data and ran end-to-end queries. Several constraint errors and cardinality issues were caught only during query development, not during schema design.
- **Queries should drive schema design.** Defining the target queries and user stories earlier in the project would have reduced rework. Some late schema changes were necessary because a planned query could not be expressed with the existing table structure.
- **PostgreSQL-specific features added real value.** Constructs such as `FILTER`, `GREATEST`/`LEAST`, materialized views, and window functions made the SQL considerably more expressive than standard SQL alternatives. Learning these features was worth the initial investment.

5 Conclusion

When we started this project our goal was to improve the course registration system, and creating an all-in-one database where students can find their classes, schedules, instructors and learning outcomes in one place, without having to navigate through multiple websites and the hurdles that come along with it. To that extent we have met our goal.

We developed a working PostgreSQL prototype for student registration by bringing together fragmented course, program, instructor, student, and timetable data into one relational schema; the aim of the project is to show how a unified database can improve both data integrity and usability.

The implemented queries and views demonstrate practical uses of the system, including helping students find courses, supporting prerequisite checks, tracking credit progress, and assisting with timetable based rescheduling. These functions were specifically chosen to highlight the usability of our database and also show its ability to support decision making rather than simply storing information.

Over the course of this project, we were able to apply and improve our knowledge of PostgreSQL, while also learning how to work with the challenges of real institutional data, particularly inconsistent formatting and missing values. During the project, we also met with members of the SITS team and the head of student records. These meetings not only helped us obtain the data needed for our work, but also gave us the opportunity to share insights into the flaws of the current CEU system.

Overall, the prototype shows that a more integrated database design could provide a strong foundation for a clearer and more efficient CEU course registration system.

References

[1] E. Lvov and T. Haltrich. *CEU Students' experience with course selection process*. Unpublished raw data, Central European University. 2025.

[2] T. Giorgobiani. *CEU Course Management Experience [Survey]*. Unpublished survey. 2026.

[3] Universitat Rovira i Virgili. *Universitat Rovira i Virgili Intranet*. Access restricted; available only with David's URV login credentials. 2026. URL: https://intranet.urv.cat/group/intranet/.

[4] Central European University. *Course Data [Data set]*. Institutional dataset. 2025.

[5] Central European University. *TimeEdit Course Data [Data set]*. Institutional dataset. 2026.

[6] Central European University. *Scheduling report 25*$_2$6[*Dataset*]. Institutional dataset. 2026.

[7] Petra Karl Novak. *Suggestion for Learning Outcomes Attributes*. Internal guidance document. 2026.

[8] Holistics Software. *dbdiagram.io*. Used to create the ER model representation in this project proposal. 2026. URL: https://dbdiagram.io/.

A SQL Listings

Listing 1: Schema excerpt: lesson table

```
create table lesson (
  lesson_id int generated always as identity primary key,
  course_id int not null references course(course_id) on delete cascade,
  class_id int not null references class(class_id) on delete cascade,
  lesson_type varchar(50),
  weekday varchar(10) not null,
  start_time time not null,
  end_time time not null,
  check (lower(weekday) in ('monday', 'tuesday', 'wednesday', 'thursday', 'friday')),
  check (end_time > start_time)
);
```

Listing 2: View: student timetable materialized view

```
create materialized view student_timetable as
select weekday, course, time, lesson_type, room
from (
  select
    lower(l.weekday) as weekday,
    c.course_code || ' - ' || c.course_name as course,
    l.start_time::text || ' - ' || l.end_time::text as time,
    l.lesson_type,
    cl.class_name as room
  from student_enrolled_in_course sec
  join student s on s.student_id = sec.student_id
  join course c on c.course_id = sec.course_id
  join lesson l on l.course_id = c.course_id
  join class cl on cl.class_id = l.class_id
  where s.student_id = 2
) timetable_rows;
```

Listing 3: Query 1: available courses for a student

```
SELECT c.course_code, c.course_name, c.ects_credits, t.term_name,
       'Mandatory' AS course_type
FROM student s
JOIN program_mandatory_course prc ON s.program_id = prc.program_id
JOIN course c ON prc.course_id = c.course_id
JOIN term t ON c.term_id = t.term_id
WHERE s.student_id = 201
  AND c.course_id NOT IN (SELECT course_id
                          FROM student_passed_course
                          WHERE student_id = s.student_id);
```

Listing 4: Query 2: prerequisite decision support

```
SELECT s.student_first_name || ' ' || s.student_last_name AS student_name,
       c.course_code,
       CASE
         WHEN COUNT(chpc.hard_prerequisite_course_id) = 0 THEN 'AUTO-APPROVE'
         WHEN COUNT(spc.course_id)+COUNT(seic.course_id)
              = COUNT(chpc.hard_prerequisite_course_id) THEN 'AUTO-APPROVE'
         ELSE 'FLAG: prerequisites not met'
       END AS decision
FROM student_requested_enrollment_in_course req
JOIN student s ON req.student_id = s.student_id
JOIN course c ON req.course_id = c.course_id
LEFT JOIN course_has_hard_prerequisite_course chpc
  ON chpc.course_id = c.course_id
LEFT JOIN student_passed_course spc
  ON spc.student_id = req.student_id
 AND spc.course_id = chpc.hard_prerequisite_course_id
LEFT JOIN student_enrolled_in_course seic
  ON seic.student_id = req.student_id
 AND seic.course_id = chpc.hard_prerequisite_course_id
GROUP BY req.student_id, req.course_id, s.student_id, c.course_code;
```

Listing 5: Query 3: credit progress for one student

```
SELECT s.student_first_name || ' ' || s.student_last_name AS student_name,
       p.program_name,
       SUM(c.ects_credits) AS total_mandatory_ects,
       COALESCE(SUM(c.ects_credits)
               FILTER (WHERE spc.student_id IS NOT NULL), 0) AS passed_mandatory_ects,
       SUM(c.ects_credits)
         - COALESCE(SUM(c.ects_credits)
                   FILTER (WHERE spc.student_id IS NOT NULL), 0) AS remaining
FROM student s
JOIN program p ON s.program_id = p.program_id
JOIN program_mandatory_course prc ON p.program_id = prc.program_id
JOIN course c ON prc.course_id = c.course_id
LEFT JOIN student_passed_course spc
       ON spc.student_id = s.student_id
      AND spc.course_id = prc.course_id
WHERE s.student_id = 201
GROUP BY s.student_id, s.student_first_name, s.student_last_name,
         p.program_name;
```

Listing 6: Query 4 (summarized): common free time windows

```sql
WITH params AS (
  SELECT 1::int AS course_id
),
weekday_order AS (...),
course_students AS (...),
occupied_lessons AS (...),
running_busy AS (...),
busy_groups AS (...),
merged_busy AS (...),
day_bounds AS (...),
busy_with_sentinel AS (...),
free_slots AS (...)
SELECT weekday, free_start, free_end, free_duration, rooms_available
FROM free_slots
WHERE free_start < free_end
ORDER BY weekday_number, free_start;
```